EXPERT PARALLELISM ON A LAPTOP: RUNNING A 754B-PARAMETER MOE IN 24 GB

Haorui Li
Southeast University
haoruileee@gmail.com

https://github.com/haoruilee/DeepInfer-Mac

ABSTRACT

DeepSeek serves its 754B-parameter DeepSeek-V4.1-Flash model with expert parallelism. The experts of each layer are spread over many GPUs, every token is sent to the GPUs that hold its experts, and communication is hidden behind computation. We ask what remains of this design on a single laptop with 24 GB of memory. Our system, DEEPINFER-MAC, keeps the dense weights in memory and leaves the experts on the internal SSD. After routing, it reads each missing expert straight into a preallocated GPU buffer. Decoding a token then becomes a chain in which the router must finish before the reads can start, and the next layer must wait for the experts. The SSD therefore sits idle while the GPU works, and the cluster's overlap techniques recover little of that time. Prefetching the next layer's experts even makes decode slower, because the guesses delay the reads the token is waiting for. We fix this with idle-window speculation, which issues the guesses only after the current layer's reads have landed. This simple change makes decode 11–27% faster across all cache sizes we tested and leaves the output unchanged. On a MacBook Air, DEEPINFER-MAC decodes 1.23 tokens per second while keeping 9 GB free for macOS, and it stays close to DeepSeek's own deployment on open-ended text. We also show that the model's sparse-attention indexer can use BF16 matrix units without changing its scores, which roughly halves its cost in long prefill.

1 INTRODUCTION

Mixture-of-experts models activate only a small part of their parameters for each token (Fedus et al., 2022; DeepSeek-AI, 2024). DeepSeek-V4.1-Flash takes this far: it has 754B parameters, but every token uses only six of the 384 experts in each layer (DeepSeek-AI, 2026b). In the data center, DeepSeek serves the model with expert parallelism (DeepSeek-AI, 2026a). The experts are spread over many GPUs, each token travels to the GPUs that hold its experts, and two techniques keep the hardware busy. One overlaps the communication of a micro-batch with the computation of another, and the other replicates experts that many tokens use.

We wanted to know whether the same model can run on a MacBook Air with 24 GB of memory, and which of these techniques still help there. The converted model is far larger than the memory, but a single token touches only a small part of it. The attention layers, routers and other dense parts fit in a third of the memory, and the experts that one token needs are only a few gigabytes. The laptop also has a device that can hold the rest: its internal SSD. We therefore treat the SSD as the place where experts live, the role that other GPUs play in a cluster.

DEEPINFER-MAC is built around this idea, as Figure 1 shows. When the router of a layer has chosen its experts, the engine reads each missing one from the SSD in a single call that writes directly into a slot of a GPU buffer, with no copy and no detour through the operating system's page cache. All experts of the layer then run in one gathered matrix multiplication, and recently used experts stay in memory as a cache, much as a cluster keeps copies of popular experts.

Measuring this system changed how we thought about it. A cluster has many tokens in flight, so some device always has work. A laptop decodes one token at a time, and every layer forms a strict

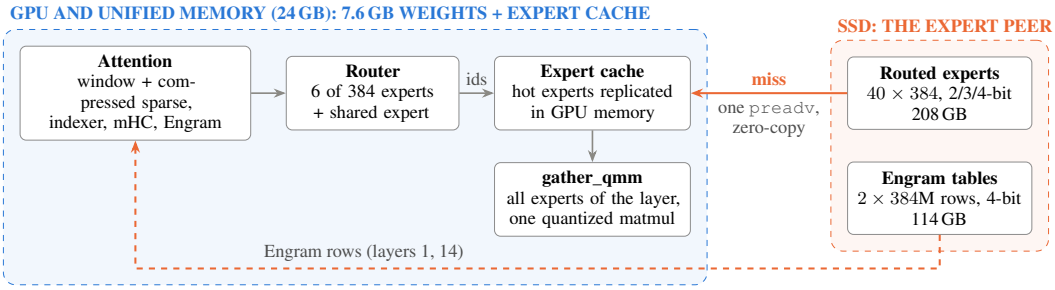


Figure 1: Overview of DEEPINFER-MAC. Dense weights and recently used experts are kept in unified memory, and missing experts are read from the SSD directly into slots of preallocated GPU buffers. Within a layer, attention, routing, reading and expert computation run one after another.

chain: attention must finish before the router knows which experts to read, and the next layer cannot start until those experts are computed. The GPU and the SSD end up taking turns. Computing the cached experts while the missing ones load, the closest analogue of micro-batch overlap, hides only a few milliseconds per layer. Prefetching the next layer's experts should use the idle SSD, as earlier work on expert offloading suggests (Eliseev & Mazur, 2023), yet in our measurements it made decode slower. An ablation shows the reason. The guesses enter the same queue as the reads the token is waiting for, and every wrong guess delays one of them.

The fix is to wait. Idle-window speculation issues the guesses only after the current layer's reads have arrived, so they run while the GPU computes and never stand in front of a needed read. It also places guessed experts where the cache evicts first. This change, about forty lines of code, turns a slowdown into an 11–27% speed-up across every cache size we measured, and traces confirm that the SSD now works during time it used to spend idle.

The engine follows the official reference implementation operation by operation, including the model's compressed sparse attention, its Engram memory, Hyper-Connections (Zhu et al., 2025), the vision encoder and the rounding steps used in training. A test suite runs DeepSeek's PyTorch code as an oracle, so that no speed-up can silently change the model.

In summary, we make the following contributions:

- We map expert parallelism to a laptop with one GPU and one SSD, and build an engine that runs every component of DeepSeek-V4.1-Flash within 24 GB with parity tests against the reference.
- We describe the critical path of a decoding token, fit a simple cost model to it, and measure parallelism at six levels of the system. Only work that fills the SSD's idle time without delaying needed reads pays off.
- We show why next-layer expert prefetching loses on a memory-bound laptop and how idle-window speculation, an application of a known prefetch-control idea, turns it into a gain.
- We prove that the model's sparse-attention indexer can run on BF16 matrix units with unchanged scores under a mild condition on its scales, and we measure exact Top-K screening in the style of DoSA (Meng & Fang, 2026) on this model, where it saves little.

On a MacBook Air with an M4 chip, DEEPINFER-MAC decodes 1.23 tokens per second while leaving 9 GB of memory to macOS, agrees with DeepSeek's API on 86% of open-ended continuation tokens, and answers a long-context retrieval question correctly at every tested length. The code, converter, tests and measurement logs are open source.

2 BACKGROUND AND MOTIVATION

2.1 DEEPSEEK-V4.1-FLASH

Table 1 summarizes the model. Every layer contains a mixture-of-experts block in which a router picks a few routed experts for each token, next to one shared expert. Attention combines a short

Table 1: DeepSeek-V4.1-Flash and the target machine.

Parameters	754B in 40 layers, hidden size 5120
Experts per layer	384 routed, 6 selected per token, plus 1 shared
Attention	128-token window and compressed sparse attention
Indexer	32 heads of dimension 128, keeps 512 positions per query
Candidate selection	2048 blocks of 8 positions, chosen at layer 20
Engram	layers 1 and 14, two tables of about 384M rows
Release size	510 GB, FP8 dense weights and FP4 experts
Machine	MacBook Air, Apple M4 with a 10-core GPU
Memory	24 GB unified, shared by CPU and GPU
Storage	1 TB internal SSD, about 3 GB/s for large reads

sliding window with *compressed sparse attention*. Keys are compressed along the sequence, and a small multi-head indexer (DeepSeek-AI, 2025) chooses which compressed positions each query attends to. The selection works on two levels: one layer picks candidate blocks of positions, and the indexers of later layers only score keys inside those blocks. A few layers own the compressed caches and the others reuse them. Two layers add an *Engram* module that looks up embeddings of hashed n-grams in very large tables, and the residual stream is mixed by Hyper-Connections with Sinkhorn-normalized matrices (Zhu et al., 2025). The release also includes draft layers for speculative decoding, called DSpark, and a vision encoder.

The released weights are stored in FP8 for the dense layers and FP4 for the experts. The reference code also rounds activations to FP8, FP4 or BF16 at fixed points during inference. These roundings matter most where the model makes discrete choices, because a score that rounds differently can change which keys the indexer keeps or which experts the router picks.

2.2 Expert parallelism in the data center

DeepInfer (DeepSeek-AI, 2026a), DeepSeek's inference engine, serves this model on a GPU cluster with expert parallelism. Each GPU stores a subset of the experts. After routing, an all-to-all *dispatch* sends every token's hidden state to the GPUs that hold its experts, and a *combine* step returns the weighted results. DeepSeek uses two further techniques to keep the GPUs busy (DeepSeek-AI, 2024). The computation of one micro-batch overlaps with the communication of another, which hides the all-to-all, and frequently used experts are copied to several GPUs, which spreads the load. Both techniques rely on having many tokens in flight at once.

2.3 Why a laptop might be enough

The converted model is more than ten times larger than the laptop's memory, but one decoding step touches far less. The dense part of the model fits in about a third of the memory. The experts that a single token needs add up to a few gigabytes, and if half of them are already cached, the SSD has to deliver less than two gigabytes per token. At the drive's speed, that is well under a second. A token per second therefore looks reachable, provided that the dense weights stay resident, reads go straight into GPU memory, and the cache gets every byte that is left.

Seen this way, the SSD takes the place that other GPUs hold under expert parallelism. This raises the question we study in the rest of the paper: *which parts of expert parallelism still help when there is one GPU, one SSD and one sequence?*

3 Design

Figure 1 gives the overall design, and Table 2 lists how each part of expert parallelism carries over. This section first describes how an expert travels from the SSD to the GPU. It then follows one decoding token through a layer to find where time is lost, and ends with the two mechanisms that win it back: idle-window speculation, which is new, and an expert cache that takes the place of expert replication.

Table 2: Expert parallelism on a GPU cluster and the corresponding mechanism in DEEPINFER-MAC.

	Expert parallelism on a cluster	DEEPINFER-MAC
Where experts are stored	GPU memory of many devices	records on the SSD
After routing	dispatch tokens to experts	read missing experts into GPU buffers
Expert computation	grouped GEMM on each GPU	one `gather_qmm` over buffer slots
Hiding transfers	overlap two micro-batches	read ahead while the GPU computes
Load balancing	replicate hot experts	LRU expert cache in unified memory

3.1 READING EXPERTS INSTEAD OF SENDING TOKENS

In a cluster, tokens travel to the experts. On a laptop the token already sits next to the only GPU, so the expert has to travel instead, and every copy along the way costs memory that the cache could use.

Records and slabs. Each expert is stored on the SSD as one page-aligned record that holds the packed weights, scales and biases of its three projections, laid out exactly as MLX (Hannun et al., 2023) lays out quantized tensors in memory. At start-up the engine allocates a *slab* for each precision layout: three GPU arrays whose leading dimension indexes slots, one expert per slot. On a cache miss the engine claims a free slot and issues a single `preadv` call whose buffers point into the three arrays at that slot. Because CPU and GPU share memory, the GPU later reads the bytes exactly where the SSD put them. The files are opened with `F_NOCACHE`, so macOS does not keep a second copy in its page cache.

One matmul per projection. When the router of a layer has finished, the host reads the chosen expert ids. This is the only point in a layer where the CPU waits for the GPU. The engine maps each expert to its slot and pins the slot until the layer is done. All token-expert pairs of the layer then go through one `gather_qmm` call per projection, with the activation function and the routing weights fused into the same compiled function. This step plays the part of the combine step in a cluster, without any communication.

3.2 THE CRITICAL PATH OF ONE TOKEN

Figure 2a shows four consecutive layers of a real decoding step. The GPU first runs attention and the router. Only then does the host know which experts the token needs, so only then can the missing ones be requested from the SSD. The GPU computes the experts once they have arrived, and the next layer cannot begin before that. The GPU and the SSD therefore take turns, and each spends much of the step waiting for the other.

This chain suggests a simple cost model. Let a be the time a layer spends on attention and routing, c the time spent computing experts, m_ℓ the number of experts missing in layer ℓ, b the size of one expert and B the bandwidth of the SSD. A decoding step then takes about

$$t_{\text{token}} \;\approx\; \sum_{\ell=1}^{40} \Big(a + c + \frac{m_\ell \, b}{B} \Big). \tag{1}$$

On our machine the reading term is about two thirds of a step, so the model points to three ways of making a step shorter. The first is to have fewer misses, which needs a larger cache. The second is to overlap reads with computation that does not depend on them, but inside a layer that is only the computation of already cached experts, which is short. The third is to read experts before the router asks for them, so that the SSD also works while the GPU computes. Section 6.2 measures each of these options against the others.

3.3 IDLE-WINDOW SPECULATION

Equation (1) also shows when the SSD is idle: whenever the GPU runs attention or computes experts. Over a full step this idle time would be enough to read dozens of experts. Prefetching experts is a

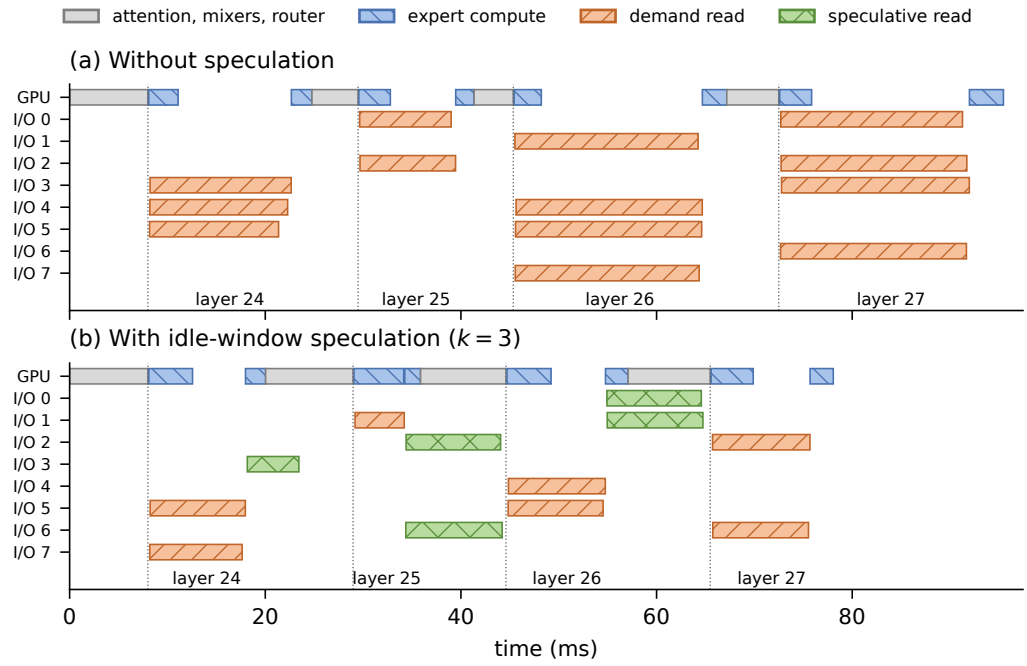


Figure 2: Measured timeline of four decode layers with a 5 GB expert cache. Gray bars are attention and routing, blue bars are expert computation, orange bars are demand reads and green bars are speculative reads, one row per I/O thread. In (a) the reads of a layer start only after its router finishes, and the SSD is idle while the GPU works. In (b) idle-window speculation reads guesses for the next layer during that idle time, so several of its experts are already in memory when its router finishes. Each panel shows the median of three traced tokens. Timing the GPU work requires a synchronization after each expert group, which slows the traced steps by 2% in (a) and 14% in (b).

well-known idea (Eliseev & Mazur, 2023; Hwang et al., 2024; Xue et al., 2024a; Song et al., 2024a; Zhong et al., 2024), and Eliseev & Mazur (2023) already guess the next layer's experts by applying that layer's gate to the current hidden state. We used the same predictor, yet our first prefetcher made decode slower. The traces suggested two causes, and our ablation later separated them.

The first cause is ordering. The guesses for the next layer were issued together with the current layer's misses and shared the same I/O queue, so a wrong guess could hold up a read that the token was waiting for. The second possible cause is cache pollution. A guessed expert entered the cache as if it had just been used, so a wrong guess could push out an expert that the model needed again soon.

Idle-window speculation addresses both, and it needs no training and no change to the model. When the router of a layer has finished, the engine also applies the next layer's router, with its exact scoring, to the current hidden state and keeps the top three experts as guesses. A separate thread holds these guesses back until every miss of the current layer has arrived, and only then issues their reads. The guesses therefore travel while the GPU computes, in the window where the SSD would otherwise be idle, and they never stand in front of a needed read. In addition, a guessed expert enters the cache at the end that is evicted first, similar to low-priority insertion in processor caches (Qureshi et al., 2007). If the next layer uses it, it moves to the other end like any hit; if not, it is the first entry to go. A guess can never evict an expert that the current layer is using. The whole change is about forty lines in the expert store and the mixture-of-experts block. The ablation in Section 6.2 shows that waiting for the demand reads provides almost all of the gain, while cold insertion adds only a little. Figure 2b shows the effect on the same four layers: the guesses arrive while the GPU works, and the demand reads that remain are fewer and shorter.

Table 3: Converted weights of DeepSeek-V4.1-Flash as stored on disk. Sizes are in GiB, which we write as GB throughout.

Component	Format	Size	Location
Dense layers: attention, shared experts, routers, mHC, Engram projections	`mxfp8`, lossless	6.9	memory
Output head and other global tensors	affine 8-bit / BF16	0.7	memory
Embedding	BF16, memory-mapped	1.2	SSD
Routed experts, 40×384	affine 2/3/4-bit, $g = 64$	208.3	SSD
DSpark layers: dense parts / experts (3×128)	`mxfp8` / affine 2/3-bit	0.7 / 4.5	SSD
Vision encoder	BF16	0.9	SSD
Engram tables, 2×384M rows	affine 4-bit, $g = 32$	114.4	SSD
Total on disk		337.6	

3.4 Replication: spending memory on the cache

Copying popular experts is the part of expert parallelism that carries over most directly. A cluster replicates experts so that no GPU becomes a bottleneck; we keep recently used experts in memory so that no read is needed at all. Each hit removes one read from the chain in Equation (1). We use least-recently-used replacement, which beat a frequency-based policy when we replayed routing traces from the real model. The replays also showed that the size of the cache matters much more than the replacement policy.

Because the size of the cache governs decode speed, the rest of the design saves memory wherever it can. Dense FP8 weights convert to MLX's MXFP8 format without loss, so no higher-precision copy is kept. Expert reads bypass the page cache, so no expert is stored twice. Speculative reads enter the cache cold, and speculative decoding is off by default, so neither pushes useful experts out. At start-up the cache takes the memory that is free at that moment, minus a safety margin, and it always leaves at least 9 GB to macOS. Below that, the system starts to swap and every gain is lost.

The same reasoning motivates an optional approximation. When a missing expert carries only a small share of a token's routing weight, the engine can skip it and save the read. This costs some agreement with the API on open-ended text, so it is off by default, and we did not include it in the speed measurements.

4 Implementation

DeepInfer-Mac is about 3,800 lines of Python on top of MLX (Hannun et al., 2023), plus one Metal kernel that Section 5 describes. We ported the official reference implementation operation by operation, because we wanted every later speed-up to be checkable against it. The engine is not lossless as a whole, since the experts are compressed, but everything else is meant to compute exactly what the reference computes.

Weight format. Table 3 shows where each part of the model ends up. The FP8 dense weights convert to MLX's MXFP8 format without any loss, because both formats store the same E4M3 values and differ only in how they share scales. The experts are requantized from FP4 to two to four bits per weight. For every group of weights the converter searches for the clipping ratio with the smallest reconstruction error, and it chooses the bit width of each layer by comparing hidden states against the original experts, as the appendix describes. Lossless MXFP4 experts would not have fit on our disk next to the download stream, but they are a one-line change for machines with more storage. The Engram tables are stored as 4-bit rows and stay within a fraction of a percent of the originals.

One forward path. As in DeepInfer, prefill, decode, chunked prefill, prefix reuse and speculative verification all share one forward function that can process any chunk of tokens at any position. The state of a sequence, which includes the sliding-window buffers, the compressed caches and the compressor's unfinished tail, can be saved and restored exactly. Two features build on this. With *bounded replay*, the later half of the layers processes only the last 140 tokens of a prompt, because

those layers see older tokens only through compressed caches that earlier layers have already filled. This roughly halves the expert reads of a long prefill. With *conversation reuse*, the engine keeps the last sequence and a snapshot taken just before the end of the last prompt, so a follow-up turn only processes its new tokens. The snapshot matters in thinking mode, where the chat template removes the previous turn's reasoning from the next prompt.

Other components. Engram rows are read from the SSD through a row cache. DSpark drafts are verified with speculative rejection sampling, which keeps the output distribution exact, and a small controller turns speculation off when it is slower than plain decoding. The server speaks the OpenAI, Anthropic and Responses APIs and reuses DeepSeek's own prompt rendering and output parsing (DeepSeek-AI, 2026c), while structured outputs use XGrammar (Dong et al., 2024). The converter never stores the 510 GB release: it reads each tensor from the model hub with range requests, converts it on the GPU, writes it to its final place, and can resume after any interruption.

Testing. The test suite runs DeepSeek's PyTorch model, with CPU versions of its GPU kernels, on small random configurations that exercise every mechanism of the model. With activation rounding switched off on both sides, prefill and decode logits agree with a median cosine above 0.9999, the vision encoder matches exactly, and greedy speculative decoding produces the same tokens as greedy decoding. On the real checkpoint, every converted layer agrees with the reference on a real chat prompt, and the Engram hash ids are identical.

5 Case study: the sparse-attention indexer

Long prompts take minutes on a laptop, and part of that time goes to the indexer. For a query with indexer heads $\mathbf{q}_1, \dots, \mathbf{q}_H \in \mathbb{R}^d$ and signed head weights $w_1, \dots, w_H$, the reference scores each visible compressed key $\mathbf{k}_j$ as

$$s_j \;=\; \mathrm{bf16}\Big(\sum_{h=1}^{H} \mathrm{bf16}\,\big(\big[\mathrm{bf16}(\mathbf{q}_h^\top \mathbf{k}_j)\big]_+ \, w_h \big) \Big), \tag{2}$$

with the outer sum accumulated in FP32, and keeps the best K keys. Prefill scores every query against every visible key, so this cost grows with the square of the prompt length. We tried two ways to make it cheaper. The first computes Equation (2) faster without changing any result, and the second skips keys that cannot make the cut.

5.1 Faster scoring with the same results

BF16 products. The reference computes each product $\mathbf{q}_h^\top \mathbf{k}_j$ in FP32 and then rounds it to BF16. Both inputs are FP4 values with power-of-two scales, which BF16 represents exactly, and the following lemma shows when a BF16 matrix multiplication returns the same rounded result.

Lemma 1. *Let $\mathbf{q}, \mathbf{k} \in \mathbb{R}^{128}$ hold E2M1 values with one power-of-two scale per block of 32 elements, and let σ_b be the product of the two scales of block b. Assume that $\max_b \sigma_b / \min_b \sigma_b \le 2^9$ and that every block product stays in the normal FP32 range. Then every partial sum of $\mathbf{q}^\top \mathbf{k}$, in any order, is exactly representable in FP32, and a BF16 matrix multiplication that accumulates in FP32 and rounds once returns $\mathrm{bf16}(\mathbf{q}^\top \mathbf{k})$, the reference result.*

The appendix gives the proof. MLX's BF16 matrix kernels accumulate in FP32, although we checked this empirically rather than for every kernel variant. Where the scale condition fails, both computations are ordinary FP32 sums whose last bit can depend on the order of summation, just as it can between different GPU kernels running the reference. On indexer inputs captured from a real 32K-token prompt, the BF16 version reproduced every one of about a billion scores.

Fused head reduction. After the matrix multiplication, Equation (2) still needs a ReLU, a weighting and a rounding for every element, followed by a sum over heads. Written as array operations, this reads a large intermediate tensor three times. We replaced it with a single Metal kernel in which each thread handles one pair of query and key and walks over the heads, applying the same roundings in the same order. On every input we tested it matches the array version. Both sum the heads in

FP32, so their results could differ in the last bit only if the summation orders differed and the terms spanned a very wide range, which is equally true of the reference. Figure 3a shows that together the two changes make an indexer block about twice as fast on real inputs, with the same selected keys.

Fixed tie order. A partition-based Top-K leaves ties at the boundary to the kernel. We sort by a 64-bit key built from the BF16 score and the position, so that ties go to the earlier position and a NaN score is never selected. Selection thereby becomes a pure function of the scores, which an exact screening method must also reproduce.

5.2 Skipping keys: exact screening

DoSA (Meng & Fang, 2026) skips keys while keeping the selection exact. A seed set of keys gives a lower bound τ on the K-th best score, a cheap upper bound U_j is computed for every other key, and keys with $U_j < \tau$ are dropped before full scoring. DoSA reports large speed-ups on GLM-5.2 and DeepSeek-V3.2 at contexts of half a million tokens and more. V4.1-Flash differs in three ways that could matter: it has fewer indexer heads, a smaller K, and its own two-level candidate selection. We therefore measured how well such a bound works on this model.

Setup. We captured the indexer inputs of a real 32K-token prompt and studied every index layer except the one that chooses the candidate blocks, which needs all block maxima and cannot skip keys. For each layer we used the last decode query and sixteen queries from the prompt. A key is *eligible* for a query if the query can see it and, in the later layers, if it lies inside the query's candidate blocks. As in DoSA, the bound evaluates the positive heads P and the strongest negative heads E exactly, and merges the remaining negative heads into a single term:

$$U_j = \sum_{h \in P} w_h \left[\mathbf{q}_h^\top \mathbf{k}_j\right]_+ - \sum_{h \in E} |w_h| \left[\mathbf{q}_h^\top \mathbf{k}_j\right]_+ - \left[\sum_{h \in N \setminus E} |w_h| \, \mathbf{q}_h^\top \mathbf{k}_j\right]_+ . \tag{3}$$

The bound is valid because $\sum_i [x_i]_+ \geq [\sum_i x_i]_+$. We enlarged it slightly to cover the BF16 roundings of Equation (2) and checked that no dropped key belongs to the true Top-K. Its cost counts the fully scored seed keys, the dot products of the bound for every other eligible key, and the remaining heads for every key that survives.

Results. Figure 3b and the appendix show the cost at the best head budget for each layer. A useful bound needs about two thirds of the heads, and even then screening saves only a tenth to a fifth of the indexer's work, before the overhead of compacting the survivors on the GPU. DoSA's joint bound for omitted positive heads did not drop a single additional key on this model, and a low-rank bound built from the leading singular vectors of the query heads failed as well, because those singular values decay slowly. Two properties of the model explain the difference from DoSA's setting. At these context lengths, K is a large fraction of the keys, so the threshold τ sits low in the score distribution. And the model's own candidate selection already removes many keys before the indexer score matters. Since the indexer is only a small part of a 32K-token prefill, the end-to-end gain would be around one percent. We left screening off and shared the measurements with the DoSA authors. The picture may change at contexts of 128K tokens and more, but prefilling such a prompt takes hours on this machine.

6 Evaluation

Setup. All experiments run on the MacBook Air of Table 1 with other applications closed. For the main decode experiments, all compared settings run interleaved in one process, so that they see the same cache state and the same background load. Each block decodes six tokens with one setting, the order of settings rotates every round, and we use three prompts in English and Chinese. We report the mean over blocks of the median time per token, with 95% confidence intervals from Student's t distribution, and we compare every block with the block of the reference setting from the same round. The evaluation answers three questions: how fast the system is, which kinds of parallelism help, and how close its output is to DeepSeek's own deployment.

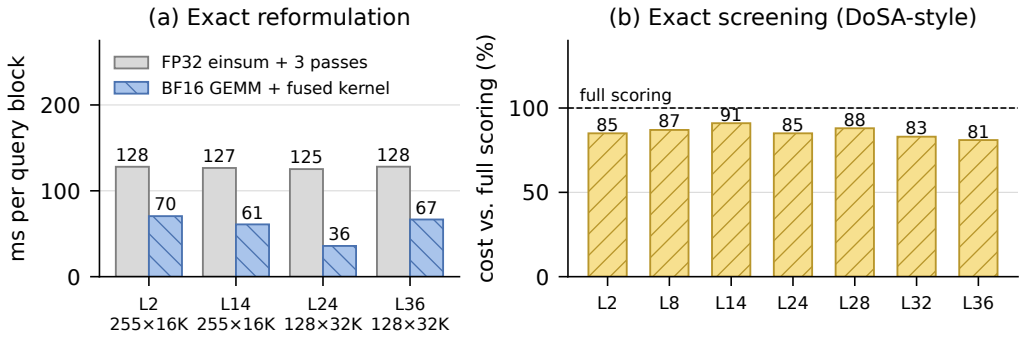


Figure 3: (a) Indexer time per prefill query block on real inputs, before and after the BF16 matmul and the fused head reduction. Labels give the layer and the block shape as rows times keys. Scores and selected keys are identical on these blocks. The matmul alone runs 1.5 to 1.9 times faster and the head reduction 5 to 6 times faster. (b) Cost of an exact screening bound relative to fully scoring the eligible keys, per index layer, at the best head budget. The bound was checked offline in float64 with slack for the BF16 roundings; the cost includes scoring the seed keys and is averaged over 17 queries of a 32K-token prompt with $K = 512$.

Table 4: End-to-end results on a MacBook Air with 24 GB. Decode uses idle-window speculation unless noted. Quality numbers come from teacher forcing against continuations sampled from DeepSeek's API.

Decode, 5.5 GB expert cache, 9 GB left to macOS	1.23 tokens/s
Decode, 5.5 GB expert cache, without speculation	1.08 tokens/s
Decode, 6.5 GB expert cache	1.35 tokens/s
Speed-up from speculation, 2.5 to 6.5 GB caches	11–27%
First token for a 14-token, 1K-token and 16K-token prompt	10 s, 49 s, 9 min
SSD read bandwidth with one or several reads in flight	2.7, 3.0 GB/s
Top-1 agreement on 129 open-ended continuation tokens	86.0%
Mean top-1 agreement over all 17 cases	96.7%
Mean NLL per token, ours and DeepSeek's API	0.127, 0.046
Needle retrieval, 12 lengths up to 16,385 tokens	12 of 12 correct

6.1 Speed and the cost model

Table 4 summarizes the end-to-end results. With an expert cache that still leaves 9 GB to macOS, DeepInfer-Mac decodes 1.23 tokens per second. Figure 4 shows how decode speed depends on the size of the cache. Speed rises steadily as the cache grows, because every additional hit removes a read from the chain. Idle-window speculation shifts the whole curve upward. It is faster at every cache size, in 43 of 45 paired blocks, and it cuts the time spent waiting for the SSD by roughly a quarter.

The cost model of Equation (1) describes the runs without speculation closely. A two-parameter fit to the bytes each token has to read matches all five cache sizes within two percent. Applied to the runs with speculation, the same fit predicts that they should be even faster than they are, because it ignores the speculative reads. Those reads are not free: about half of their volume still costs time, and the other half falls into time when the SSD would have been idle. The traces confirm this directly. Without speculation the SSD is busy for less than two thirds of a decode step, and with speculation the busy share rises by roughly a fifth, almost all of it taken up by reads that overlap no demand read.

Prefill. Prefill behaves differently from decode. Even a short prompt routes to most experts of every layer, so the first chunk reads nearly the whole expert set once, and long prompts are limited by GPU compute rather than by the SSD. Bounded replay keeps the later layers out of most of

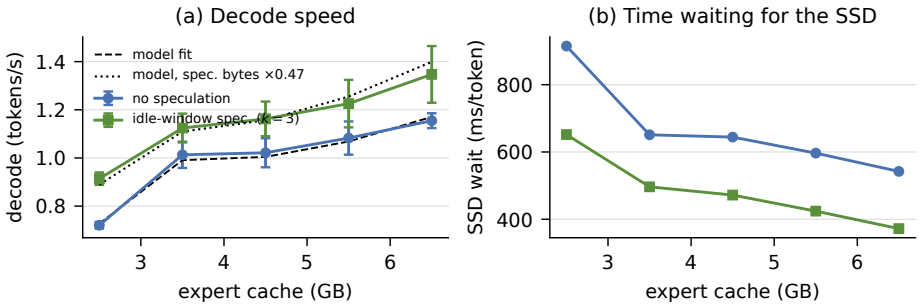


Figure 4: (a) Decode speed against expert cache size, with and without idle-window speculation, interleaved in one process with 95% confidence intervals over nine blocks. The dashed line is the cost model $t = C + D/B$ fitted to the runs without speculation, with $C = 117\,\text{ms}$ and $B = 2.37\,\text{GB/s}$, which matches them within 2%. The dotted line adds the speculative bytes with weight $\alpha = 0.47$, fitted on the same runs. (b) Time per token spent waiting for the SSD.

Table 5: Decode schedules at a 5 GB cache, interleaved in one process. Speed-ups are paired against the serial schedule of the same round, with 95% confidence intervals over 12 blocks. All speculative schedules read three guesses per layer. In the first run the naive prefetcher ranks its guesses by the raw gate logits; the second run repeats the key schedules with the router's exact scoring for every prefetcher. Tokens per second and SSD wait are from the first run.

Schedule	Guesses wait	Guesses cold	Tokens/s	SSD wait, ms	Speed-up, run 1	Speed-up, run 2
Serial			1.03	630	1.00	1.00
Overlap cached and missing experts			1.07	617	1.05 ± 0.04	
Naive next-layer prefetch (Eliseev & Mazur, 2023)	no	no	0.97	667	0.95 ± 0.07	0.89 ± 0.05
Guesses issued with the demand reads	no	no	0.99	637	0.96 ± 0.05	
Guesses inserted cold only	no	yes	0.87	770	0.85 ± 0.06	
Guesses wait for demand reads only	yes	no	1.17	464	1.15 ± 0.05	1.10 ± 0.05
Idle-window speculation	yes	yes	**1.20**	**456**	$\mathbf{1.17 \pm 0.05}$	$\mathbf{1.14 \pm 0.05}$

that work. We prefill in chunks of 4K tokens, because a single long chunk is no faster and needs considerably more memory at its peak.

6.2 Where does parallelism help?

We now return to the question of the introduction and look at each level of the system where parallelism could help. Table 5 and Figure 5 show the interleaved ablation of the decode schedule, and Table 6 collects the other techniques we tried.

Inside the SSD. The drive itself is the one place where parallelism comes for free. It delivers expert records noticeably faster with several requests in flight than with one, and it saturates after a few, which our eight I/O threads easily provide. Splitting a single expert into smaller reads adds nothing, because a read of an entire expert is already large.

Inside a layer. Computing the cached experts while the missing ones load is the closest match to the cluster's micro-batch overlap. It helps a little. Figure 2a shows why it cannot help more: the cached experts of a layer take a few milliseconds to compute, while the reads take several times longer.

Across layers. This is where the schedule matters most. Naive prefetching of the next layer's experts is slower than not prefetching at all, in both runs of the ablation. It even collects twice as many speculative hits as idle-window speculation and still loses, because its guesses compete with the reads the token is waiting for. Making the guesses wait for those reads is what turns the loss into a gain, and it accounts for almost all of the improvement. Inserting the guesses cold adds a little on top, within the confidence interval. On its own, cold insertion is harmful, because guesses that arrive too early are evicted again before the next layer can use them. With both changes, idle-window

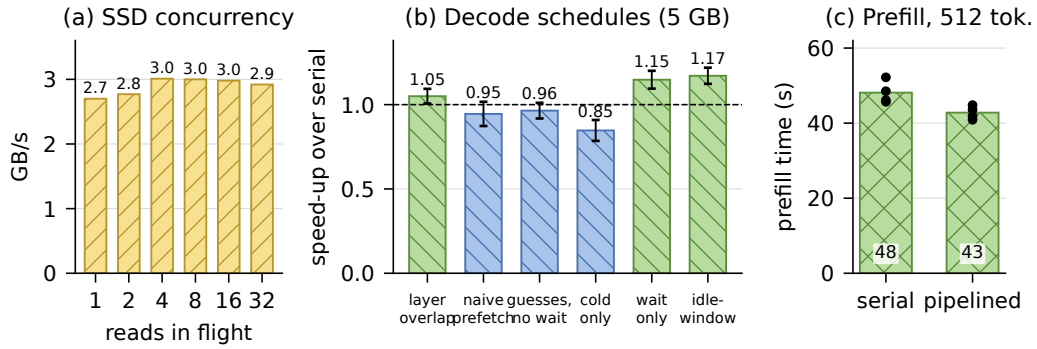


Figure 5: Parallelism on one laptop. (a) Read bandwidth for expert records against the number of reads in flight. (b) The first run of the schedule ablation in Table 5, as speed-up over the serial schedule with 95% confidence intervals; green marks schedules that are faster than serial. (c) Prefill of a 512-token prompt with and without reading the next batch of experts while the current one computes; bars show means and dots individual runs.

Table 6: Other techniques, each measured against a baseline from the same session in tokens per second, or in seconds for prefill.

Technique	Without	With	Note
Pipelined expert batches in prefill, 512 tokens	48.1 s	42.8 s	interleaved, four runs each
Four-way split reads per expert	1.12	1.11	
OS page cache behind a 5 GB slab	1.12	0.20–0.61	
DSpark speculative decoding, fixed draft length	1.28	0.35	
Two, three, four or six guesses per layer		1.13, 1.19, 1.14, 1.06	same day, not interleaved

speculation was faster than the serial schedule in 23 of 24 paired blocks across the two runs, and it produced exactly the same tokens as decoding without it. Three guesses per layer worked best; with more, the guesses again started to delay demand reads.

Across expert batches in prefill. During prefill a whole chunk of tokens goes through each layer, which gives the engine something like a batch. Reading the next batch of experts while the current one computes made prefill about a tenth faster on an otherwise idle machine. In an earlier session with other programs holding memory, the same pipelining caused swapping and gained nothing, because the extra batch needs room.

Across tokens. Speculative decoding checks several drafted tokens in one pass (Leviathan et al., 2023). On a GPU this amortizes weight reads over several tokens. Here the drafted tokens route to different experts, so the check must read the union of their experts, and decode became several times slower. The controller described in Section 4 detects this and falls back to plain decoding.

In the operating system. The page cache could in principle serve as a larger second-level cache behind the slab. In practice it competes with the slab for the same memory, and no setting we tried was faster than bypassing it.

Taken together, the measurements give a simple rule. Parallelism pays off when it gives the SSD work during time that the dependency chain would leave idle, without delaying a read the token is waiting for and without taking memory from the cache. Idle-window speculation meets all three conditions, while naive prefetching, speculative decoding and the page cache each break one of them. Even so, the size of the cache remains the strongest single factor.

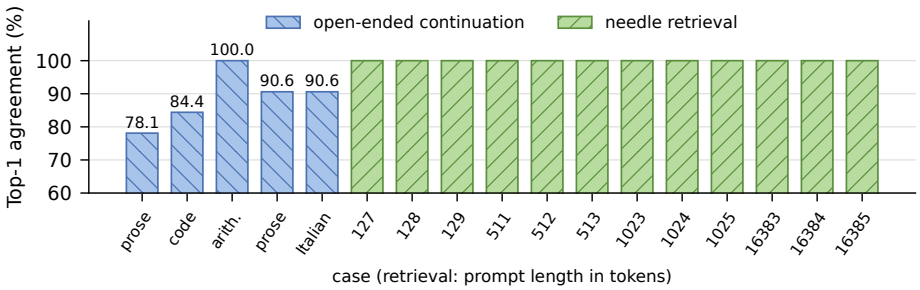


Figure 6: Top-1 agreement with continuations sampled from the official API. The retrieval prompts have one-token answers.

6.3 Output quality

We compare DeepInfer-Mac with DeepSeek's own deployment using continuations sampled from DeepSeek's API, published with the ds4 project (Sanfilippo, 2026), and run them through the converted model with teacher forcing. Figure 6 shows top-1 agreement for each case. On the five open-ended continuations, our model picks the same next token as the API for 86% of the tokens. The gap comes from compressing the experts to two or three bits, which matters more for free text than for retrieval. The other twelve cases are needle-retrieval prompts whose lengths sit just around the sliding window, the boundaries of the key selection and the candidate-block level. The model answers all of them correctly, which shows that the long-context machinery works as in the reference.

7 Related work

Running models larger than memory. FlexGen offloads weights, activations and the KV cache to CPU memory and disk, and searches for a schedule that maximizes throughput for large batches (Sheng et al., 2023). LLM in a Flash keeps weights in flash storage and loads only the rows that predicted-active neurons need, reading them in large chunks (Alizadeh et al., 2024). PowerInfer keeps frequently activated neurons on the GPU and computes the rest on the CPU (Song et al., 2024b), and llama.cpp memory-maps quantized weights and leaves residency to the operating system (Gerganov et al., 2023). For mixture-of-experts models, Eliseev & Mazur (2023) combine an LRU expert cache with speculative loading of the next layer's experts on consumer GPUs, and our predictor is theirs. Pre-gated MoE changes the model so that each layer selects the next layer's experts (Hwang et al., 2024). MoE-Infinity and ProMoE prefetch and cache experts based on activation statistics (Xue et al., 2024a; Song et al., 2024a), and AdapMoE adapts gating and prefetching to the cache (Zhong et al., 2024). Fiddler computes experts on the CPU instead of moving them (Kamahori et al., 2025), MoE-Lightning pipelines CPU and GPU work for large batches (Cao et al., 2025), and EdgeMoE and PowerInfer-2 bring expert or neuron offloading to phones (Yi et al., 2023; Xue et al., 2024b). Our setting differs from most of these in two ways. The dense part of the model alone takes a third of the memory, and the workload is a single interactive sequence. Under these conditions our measurements favour a fixed set of cache slots, filled by reads that bypass the page cache, over operating-system caching or eager prefetching.

Prefetch control. Giving demand requests priority over prefetches, and inserting prefetched data with low priority, are established techniques in processor memory systems (Lee et al., 2008; Srinath et al., 2007; Qureshi et al., 2007). Idle-window speculation applies both ideas to expert reads from an SSD. Our contribution there is the measurement showing why naive expert prefetching loses on a memory-bound laptop, and the finding that ordering, not cache pollution, is the part that matters.

Expert parallelism. DeepSeek's serving stack spreads experts over many GPUs, overlaps the all-to-all communication with computation using two micro-batches, and replicates heavily used experts

(DeepSeek-AI, 2024; 2026a). We map these ideas to a single GPU and SSD and measure which of them still apply.

DeepSeek-V4 models on Apple silicon. Several community projects run DeepSeek-V4-generation models on Macs, and we learned from all of them. ds4 is a dedicated C and Metal engine whose API-sampled test vectors we use for evaluation (Sanfilippo, 2026), flash-moe streams experts from the SSD in C and Metal (danveloper, 2026), and two MLX ports target V4-Flash and V4.1-Flash (ssd-moe, 2026; PipeNetwork, 2026). DEEPINFER-MAC differs in following the official reference implementation operation by operation, which lets us test against it, and in sharing its forward path and serving interface with DeepInfer (DeepSeek-AI, 2026a) and deepseek-recipe (DeepSeek-AI, 2026c).

Quantization. Post-training methods such as GPTQ lower weight precision with second-order error correction (Frantar et al., 2023). We use a simpler per-group clipping search, because the converter streams each expert from the network once and has no calibration pass. Instead we choose the precision of each layer separately and keep the dense weights lossless.

Sparse-attention indexers. DeepSeek Sparse Attention selects keys with a light multi-head indexer (DeepSeek-AI, 2025). V4.1-Flash adds key compression and a two-level candidate selection (DeepSeek-AI, 2026b), and IndexCache reuses indices across layers (Bai et al., 2026). Several methods speed up the selection while keeping the full scorer's output. GVR checks a predicted threshold during decoding (Cheng et al., 2026), LiteTopK fuses scoring with candidate filtering (Yin et al., 2026), and DoSA drops keys using score bounds (Meng & Fang, 2026), following the MaxScore and WAND idea from document retrieval (Turtle & Flood, 1995; Broder et al., 2003). MISA screens with a subset of heads but does not guarantee the full scorer's selection (Zhou et al., 2026).

8 LIMITATIONS AND CONCLUSION

Limitations. DEEPINFER-MAC decodes at roughly a token per second, which suits interactive use with short answers and offline jobs but not serving many users. It runs one sequence at a time, and long prompts take minutes. The compressed experts cost some agreement with the API on open-ended text, although retrieval is unaffected, and machines with more storage can keep the experts lossless. All results come from a single machine, with short prompts and short decode blocks, and we did not measure decode at long context. We also did not compare against other engines: the community projects we cite run related checkpoints with different quantization on other hardware, and porting them to this setting was beyond our scope. A Mac with a faster SSD or more memory would shift the balance between the techniques we measured. Finally, our screening study covers contexts up to 32K tokens and does not settle the question for much longer ones.

Conclusion. We asked what expert parallelism becomes on a laptop whose only other device is an SSD. Reading experts straight into fixed GPU buffers replaces the token dispatch of a cluster, and an LRU cache replaces expert replication. A single decoding token turns each layer into a chain of computing, reading and computing again, and the SSD spends a third of every step waiting. The overlap techniques of a cluster recover little of that time, and naive prefetching even makes it worse, because its guesses delay the reads a token needs. Letting the guesses wait until those reads have landed is enough to fill the idle time, and it makes decode clearly faster without changing a single output token. With these pieces, a 24 GB MacBook Air runs the 754B-parameter DeepSeek-V4.1-Flash at about 1.2 tokens per second, close to DeepSeek's own deployment. We release the engine, the converter, the tests and all measurement logs.

AI USE STATEMENT

We used generative AI tools for drafting code, experiments and this manuscript, which were reviewed and revised by the author. The author takes responsibility for the final manuscript and implementation.

REPRODUCIBILITY STATEMENT

The engine, converter and tests are released under the Apache 2.0 license at `https://github.com/haoruilee/DeepInfer-Mac`. Section 4 and the appendix describe the weight format, and `python -m deepinfer_mac convert` rebuilds the checkpoint from the public release. The parity tests in `tests/` run against the official reference implementation when its source is available through `DSV41_REF`, and are skipped otherwise.

Every measurement in the paper comes from a script in `scripts/`, and its raw output is stored in `paper/logs`. The schedule ablation and the cache sweep come from `spec_ablation.py` and `spec_sweep.py`, the timelines from `trace_decode.py`, and the SSD and prefill measurements from `bench_io_parallel.py` and `prefill_overlap.py`. Earlier decode measurements used `decode_speed.py`, `spec_ab.py` and `schedule_ab.py`, and the replacement-policy replay used `trace_policies.py`. Output quality comes from `score_api.py`, `real_check.py` and `quant_damage.py`, and the indexer study from `capture_indexer.py`, `dosa_study.py` and `bench_kernels.py`. The captured indexer inputs, about 300 MB, are not included.

One interleaved ablation run, stored as `ablation2.json`, was disturbed by an external event that slowed every setting to below a tenth of a token per second in some blocks; we repeated it and do not use it. The quality numbers come from the v0.1.1 build, whose log is `final_eval.log`; a later build scored slightly higher, as `after_score.log` shows. `paper/figures/make_plots.py` draws all figures from `paper/figures/data` and from values copied out of these logs.

REFERENCES

Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatamifard, Minsik Cho, Carlo C. Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. LLM in a flash: Efficient large language model inference with limited memory. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024. URL https://arxiv.org/abs/2312.11514.

Yushi Bai, Qian Dong, Ting Jiang, Xin Lv, Zhengxiao Du, Aohan Zeng, Jie Tang, and Juanzi Li. IndexCache: Accelerating Sparse Attention via Cross-Layer Index Reuse. *arXiv preprint arXiv:2603.12201*, 2026. URL https://arxiv.org/abs/2603.12201.

Andrei Z. Broder, David Carmel, Michael Herscovici, Aya Soffer, and Jason Zien. Efficient query evaluation using a two-level retrieval process. In *Proceedings of the Twelfth International Conference on Information and Knowledge Management*, pp. 426–434. ACM, 2003. doi: 10.1145/956863.956944.

Shiyi Cao, Shu Liu, Tyler Griggs, Peter Schafhalter, Xiaoxuan Liu, Ying Sheng, Joseph E. Gonzalez, Matei Zaharia, and Ion Stoica. MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2025.

Long Cheng, Ritchie Zhao, Timmy Liu, Mindy Li, Xianjie Qiao, Kefeng Duan, Yu-Jung Chen, Xiaoming Chen, Bita Darvish Rouhani, and June Yang. Guess-Verify-Refine: Data-Aware Top-K for Sparse-Attention Decoding on Blackwell via Temporal Correlation. *arXiv preprint arXiv:2604.22312*, 2026. URL https://arxiv.org/abs/2604.22312.

danveloper. flash-moe. https://github.com/danveloper/flash-moe, 2026.

DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024. URL https://arxiv.org/abs/2412.19437.

DeepSeek-AI. DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models. *arXiv preprint arXiv:2512.02556*, 2025. URL https://arxiv.org/abs/2512.02556.

DeepSeek-AI. DeepInfer. https://github.com/deepseek-ai/DeepInfer, 2026a.

DeepSeek-AI. DeepSeek-V4.1-Flash: Pushing the limits of KV cache compression. Technical report, DeepSeek-AI, 2026b. URL https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash/blob/main/DeepSeek_V41_Tech_Report.pdf.

DeepSeek-AI. deepseek-recipe. https://github.com/deepseek-ai/deepseek-recipe, 2026c.

Yixin Dong, Charlie F. Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. XGrammar: Flexible and efficient structured generation engine for large language models. *arXiv preprint arXiv:2411.15100*, 2024. URL https://arxiv.org/abs/2411.15100.

Artyom Eliseev and Denis Mazur. Fast inference of mixture-of-experts language models with offloading. *arXiv preprint arXiv:2312.17238*, 2023. URL https://arxiv.org/abs/2312.17238.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022. URL https://www.jmlr.org/papers/v23/21-0998.html.

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2210.17323.

Georgi Gerganov et al. llama.cpp. https://github.com/ggml-org/llama.cpp, 2023.

Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on Apple silicon. https://github.com/ml-explore/mlx, 2023.

Ranggi Hwang, Jianyu Wei, Shijie Cao, Changho Hwang, Xiaohu Tang, Ting Cao, and Mao Yang. Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *Proceedings of the 51st Annual International Symposium on Computer Architecture*, 2024. URL https://arxiv.org/abs/2308.12066.

Keisuke Kamahori, Tian Tang, Yile Gu, Kan Zhu, and Baris Kasikci. Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models. In *International Conference on Learning Representations*, 2025.

Chang Joo Lee, Onur Mutlu, Veynu Narasiman, and Yale N. Patt. Prefetch-aware DRAM controllers. In *Proceedings of the 41st Annual IEEE/ACM International Symposium on Microarchitecture*, 2008.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, 2023. URL https://arxiv.org/abs/2211.17192.

Fanxu Meng and Taosong Fang. DoSA: Dots Sparse Attention. *arXiv preprint arXiv:2610.13990*, 2026. URL https://arxiv.org/abs/2610.13990.

PipeNetwork. deepseek-v41-mlx. https://github.com/PipeNetwork/deepseek-v41-mlx, 2026.

Moinuddin K. Qureshi, Aamer Jaleel, Yale N. Patt, Simon C. Steely, and Joel Emer. Adaptive insertion policies for high performance caching. In *Proceedings of the 34th Annual International Symposium on Computer Architecture*, pp. 381–391, 2007.

Salvatore Sanfilippo. ds4. https://github.com/antirez/ds4, 2026.

Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In *International Conference on Machine Learning*, 2023. URL https://arxiv.org/abs/2303.06865.

Xiaoniu Song, Zihang Zhong, Rong Chen, and Haibo Chen. ProMoE: Fast MoE-based LLM serving using proactive caching. *arXiv preprint arXiv:2410.22134*, 2024a.

Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. PowerInfer: Fast large language model serving with a consumer-grade GPU. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, 2024b. URL https://arxiv.org/abs/2312.12456.

Santhosh Srinath, Onur Mutlu, Hyesoon Kim, and Yale N. Patt. Feedback directed prefetching: Improving the performance and bandwidth-efficiency of hardware prefetchers. In *Proceedings of the 13th International Symposium on High Performance Computer Architecture*, 2007.

ssd-moe. deepseek-v4-flash-mlx. https://github.com/ssd-moe/deepseek-v4-flash-mlx, 2026.

Howard Turtle and James Flood. Query evaluation: Strategies and optimizations. *Information Processing & Management*, 31(6):831–850, 1995. doi: 10.1016/0306-4573(95)00020-H.

Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. MoE-Infinity: Offloading-efficient MoE model serving. *arXiv preprint arXiv:2401.14361*, 2024a. URL https://arxiv.org/abs/2401.14361.

Zhenliang Xue, Yixin Song, Zeyu Mi, Le Chen, Yubin Xia, and Haibo Chen. PowerInfer-2: Fast large language model inference on a smartphone. *arXiv preprint arXiv:2406.06282*, 2024b.

Rongjie Yi, Liwei Guo, Shiyun Wei, Ao Zhou, Shangguang Wang, and Mengwei Xu. EdgeMoE: Fast on-device inference of MoE-based large language models. *arXiv preprint arXiv:2308.14352*, 2023.

Ziqi Yin, Jianyang Gao, Peiqi Yin, Jiangneng Li, and Gao Cong. LiteTopK: Exploiting the Curse of Dimensionality for a Fused Indexer-TopK Kernel in Long-Context Sparse Attention. *arXiv preprint arXiv:2607.11976*, 2026. URL https://arxiv.org/abs/2607.11976.

Shuzhang Zhong, Ling Liang, Yuan Wang, Runsheng Wang, Ru Huang, and Meng Li. AdapMoE: Adaptive sensitivity-based expert gating and management for efficient MoE inference. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design*, 2024.

Ruijie Zhou, Fanxu Meng, Yufei Xu, Tongxuan Liu, Guangming Lu, Muhan Zhang, and Wenjie Pei. MISA: Mixture of Indexer Sparse Attention for Long-Context LLM Inference. *arXiv preprint arXiv:2605.07363*, 2026. URL https://arxiv.org/abs/2605.07363.

Defa Zhu, Hongzhi Huang, Zihao Huang, Yutao Zeng, Yunyao Mao, Banggu Wu, Qiyang Min, and Xun Zhou. Hyper-connections. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2409.19606.

A WEIGHT LAYOUT

Each routed expert is one record of three projections: gate W_1 and up W_3 (2304×5120) and down W_2 (5120×2304). Each projection is stored as packed integers, per-group scales and per-group biases in MLX's affine format with group size 64, and the record is padded to a multiple of 16 KiB. Table 7 gives the bit widths per layer. We chose them with `scripts/quant_damage.py`, which runs real layers once with the converted experts and once with the original FP4 experts and compares hidden states layer by layer. The first four layers, the layers that own KV caches or index keys, and the decoder band received 3 bits, the last layer 4 bits, and the rest 2-bit gate/up projections. The DSpark draft experts use 2-bit gate/up and 3-bit down projections.

Table 7: Expert bit widths (W_1/W_3/W_2) and record sizes.

Layers	Bits	Record
0–3, 8, 14, 20–38	3/3/3	15.5 MB
4–7, 9–13, 15–19	2/2/3	12.5 MB
39	4/4/4	19.9 MB

B PROOF OF LEMMA 1

E2M1 magnitudes are $\{0, 0.5, 1, 1.5, 2, 3, 4, 6\}$, i.e., integers in $\{0, \dots, 12\}$ times 2^{-1}. In block b with scales $s_{q,b}$ and $s_{k,b}$, each product $q_i k_i$ is therefore an integer of magnitude at most 144 times $\frac{1}{4}\sigma_b$, where $\sigma_b = s_{q,b} s_{k,b}$ is a power of two. Let $\sigma_{\min}$ and $\sigma_{\max}$ be the smallest and largest block products. Every product, and hence every partial sum over any subset of the 128 terms, is an integer multiple of $\frac{1}{4}\sigma_{\min}$. Its magnitude is at most $128 \cdot 144 \cdot \frac{1}{4}\sigma_{\max} < 2^{15} \cdot \frac{1}{4}\sigma_{\max}$. Expressed in units of $\frac{1}{4}\sigma_{\min}$, a partial sum is an integer below $2^{15}\,\sigma_{\max}/\sigma_{\min} \le 2^{24}$, which FP32's 24-bit significand represents exactly. Hence every FP32 addition in any order is exact, the accumulated value equals the real dot product, and the single final rounding to BF16 equals $\mathrm{bf16}(\mathbf{q}^\top \mathbf{k})$. This assumes that the block products stay in the normal FP32 range, i.e. $\frac{1}{4}\sigma_{\min} \ge 2^{-126}$ and no overflow. The quantizer clamps each scale at 2^{-126}, so the assumption can fail for blocks of near-zero activations; on our captured inputs the scale exponents lie roughly between -8 and 4. □

C TOP-K ORDER KEY

For a BF16 score with bit pattern b (16 bits), the key $\kappa(b) = 2^{16} - 1 - b$ if the sign bit is set and $b + 2^{15}$ otherwise is monotone in the score value, after mapping -0 to $+0$. For position $j < 2^{31}$ we select the K largest values of $\kappa(b) \cdot 2^{32} + (2^{31} - 1 - j)$, a 64-bit integer, with a partition. This yields the order "higher score first, then earlier position" regardless of the partition kernel. In the candidate-block level the query's own block has score $+\infty$, which maps to the largest key after NaNs are replaced by $-\infty$, so it is always kept.

D PER-CASE API AGREEMENT

Table 8: Teacher-forced evaluation against continuations sampled from the official API. NLL is the mean negative log-likelihood per continuation token under each model.

Case	Prompt	Continuation	NLL (ours)	NLL (API)	Top-1 agreement
Prose	15	32	0.733	0.038	78.1%
Code	17	32	0.637	0.278	84.4%
Arithmetic	18	1	0.000	0.000	100%
Prose	21	32	0.244	0.115	90.6%
Italian	15	32	0.512	0.354	90.6%
Retrieval (×12)	127–16,385	1	≤0.006	0.000	100% each
Mean over 17 cases			0.127	0.046	96.7%

E SCREENING STUDY DETAILS

Table 9 lists, for each index layer, the best configuration of the bound averaged over 17 queries (the last decode query and 16 prefill queries), with a seed set of the 4096 most recent eligible keys. e is the number of negative heads evaluated exactly, m the resulting number of dot products per key (positive heads, e negative heads and the merged term; it varies slightly between queries), and the cost includes scoring the seed keys. `scripts/dosa_study.py` reproduces the table and checks that no dropped key belongs to the true Top-K.

Table 9: Best screening configuration per layer (32K-token prompt; layers 2–14 see 16K compressed keys, layers 24–36 see 32K keys, of which about 16K are eligible). Survive is the fraction of the non-seed eligible keys whose bound is not below τ.

Layer	e	m (mean)	Survive	Cost vs. full scoring
2	0	24	18.1%	85%
8	0	23	31.7%	87%
14	4	26	24.4%	91%
24	4	21	38.5%	85%
28	0	22	40.0%	88%
32	0	24	8.9%	83%
36	0	22	19.9%	81%